

RIVALYZE

AI-Powered Competitive Intelligence Platform

Software Test Plan

About Rivalyze

Rivalyze is an AI-powered competitive intelligence platform that helps users generate strategic, boardroom-ready competitive briefs. Given a company name or a product idea, Rivalyze deploys a coordinated set of AI agents to discover and validate real competitors, runs a multi-stage analysis pipeline against them, and produces a structured output containing SWOT analysis, head-to-head comparisons, and confidence-scored supporting evidence.

The platform is designed to compress work that traditionally takes a market research analyst days into a guided, automated workflow — from entering a company or domain, through competitor discovery and confirmation, to a fully assembled dashboard of evidence-linked findings. The backend is built on FastAPI and hosted on Microsoft Azure (Central India region), with an agent-orchestration pipeline that manages competitor discovery, evidence gathering, and report generation as asynchronous, trackable jobs.

About Team Argus

Team Argus is the team behind Rivalyze, formed for Simform's CodeClash 2026 hackathon. The team is responsible for the end-to-end design, development, and quality assurance of the Rivalyze platform, and owns this test plan as the reference document for how the quality of the application is verified prior to release.

Project Name	Rivalyze
Team Name	Team Argus
Event	Simform CodeClash 2026
Document Version	1.0
Prepared By	Hely Kikani – QA Trainee, Team Argus
Date	05 July 2026
Status	Draft for Review

Document Control

Version	Date	Author	Description
1.0	05/07/2026	Hely Kikani	Initial version of the Rivalyze test plan

1. Introduction

1.1 Purpose

This document defines the test plan for Rivalyze, an AI-powered competitive intelligence platform that helps users generate strategic briefs by discovering, analyzing, and comparing competitors using a set of coordinated AI agents. It describes the scope, approach, resources, and schedule of the testing activities intended for the application, and serves as the single reference point for how quality is verified before release.

1.2 Project Background

Rivalyze was built as part of the Simform CodeClash 2026 hackathon by Team Argus. The application takes a company name or idea as input, deploys AI agents to discover and validate competitors, runs a multi-stage analysis pipeline, and produces a boardroom-ready competitive brief with SWOT analysis, head-to-head comparisons, and confidence-scored evidence. The backend is built on FastAPI and hosted on Azure (Central India region).

1.3 Audience

This plan is intended for the QA team, developers, DevOps engineers, and project stakeholders involved in Team Argus, and for reviewers evaluating test coverage and quality readiness as part of the hackathon submission.

2. Scope

2.1 In Scope

The following modules and workflows are covered by this test plan:

- Hero / Landing Page — branding, navigation, hero content, CTAs, and metric widgets
- Hero Functional Flows — navigation links, theme toggle, section scrolling, and CTA routing
- Login Page (UI) — form layout, field behaviour, and visual states
- Login Functional Flows — authentication, validation, session handling, and security checks
- Brief Page — company/domain input form, suggestion chips, and onboarding stats
- Competitor Page — competitor confirmation panel and agent deployment trigger
- Discovery & Run Page — competitor mapping, competitor card management, and agent run lifecycle
- Dashboard Page — evidence-linked claims and report presentation
- Cross-cutting concerns — responsive/UI layout checks, basic security validation (SQL injection, XSS), and API contract validation for backend endpoints

2.2 Out of Scope

- Load, stress, and performance benchmarking of the AI agent pipeline
- Formal penetration testing beyond baseline OWASP-style input validation checks
- Third-party integrations not directly exposed in the application UI
- Native mobile application testing (mobile web responsiveness only)

3. Test Objectives

The testing effort for Rivalyze aims to:

- Verify that every page renders correctly and all UI elements described in the design are present and usable
- Confirm that core user journeys — from entering a company/domain to receiving a competitive brief — work end to end
- Validate form behaviour, input handling, and error states across the Login and Brief flows
- Confirm that the API layer honours its documented contract, including response schemas and job lifecycle states
- Identify and document any undocumented or inconsistent API behaviour (e.g. contract gaps) early, before they reach production
- Ensure baseline security hygiene against common attacks such as SQL injection and XSS on user-facing forms
- Confirm the application behaves predictably on both desktop and reduced/tablet-width viewports

4. Test Strategy

Testing is organised into the levels below. Each level is executed primarily through automated Playwright specs, with manual exploratory passes used to confirm visual polish and to investigate anomalies surfaced by automation.

Test Level	Purpose	Primary Tooling	Applies To
UI Testing	Verify layout, content, branding, and visual states of each page	Playwright + TypeScript	All 8 modules
Functional Testing	Verify interactive behaviour: clicks, navigation, form submission, state changes	Playwright + TypeScript	Hero Functional, Login Functional, Brief, Competitor, Discovery & Run
Validation Testing	Verify required-field checks, format validation, and inline error handling	Playwright + TypeScript	Login Functional, Brief
Security Functional Testing	Verify safe handling of malicious input (SQL injection, XSS) and generic auth error	Playwright + TypeScript	Login Functional

Test Level	Purpose	Primary Tooling	Applies To
	messaging		
API / Contract Testing	Validate request/response schemas, status codes, and async job polling behaviour against the OpenAPI spec	Playwright + Zod schemas	Backend endpoints powering Discovery & Run
Responsive / Cross-viewport Testing	Verify layout integrity on desktop and smaller supported viewports	Playwright (viewport emulation)	Discovery & Run, Hero
Smoke Testing	Verify that each page loads without errors as a final sanity check	Playwright + TypeScript	Hero, Brief, Competitor, Discovery & Run, Dashboard

4.1 Test Design Technique

- Each test case follows a consistent structure: Preconditions → Test Steps → Expected Result → Status, allowing traceability between requirement, execution, and outcome
- Positive-path and negative-path (empty fields, invalid formats, malicious input) cases are designed together for every form
- Contract tests use schema validation (Zod) rather than hard-coded field checks, so tests fail on structural drift rather than incidental value changes

5. Test Environment

Component	Details
Frontend	Rivalyze web application (staging build)
Backend	FastAPI service hosted on Azure (Central India)
Browsers	Chromium, Firefox, WebKit (via Playwright)
Viewports	Desktop (1920×1080), Tablet width (~768px)
Test Automation	Playwright + TypeScript, Zod for schema validation
CI/CD	GitHub Actions
Test Data	Sample companies (e.g. Notion, Figma, Zomato) and synthetic invalid inputs

6. Entry and Exit Criteria

6.1 Entry Criteria

- Feature build is deployed to the staging environment and is accessible
- Test cases for the relevant module are documented and reviewed

- Required test data and environment access are available
- API contract (OpenAPI spec) is available for endpoints under test

6.2 Exit Criteria

- 100% of planned test cases for a module have been executed
- All Critical and High priority test cases pass
- All identified defects are logged, triaged, and either fixed or explicitly accepted by the team
- No open Critical-severity defects remain against in-scope modules
- Test summary report is reviewed and signed off by the QA lead

7. Test Deliverables

- Test Plan (this document)
- Module-wise manual test case documentation (Rivalyze Test Case Documentation)
- Automated Playwright test suites (contract, functional, and node-boundary tests)
- Test execution summary with pass/fail status per module
- Defect / issue log (tracked in GitHub Issues)
- API contract gap notes, where actual behaviour deviates from the documented spec

8. Roles and Responsibilities

Name	Role	Responsibility
Rushabh Sorathiya	QA Engineer	Test planning, review, and execution oversight
Hely Kikani	QA Trainee	Test case design, API contract testing, execution, and reporting
Dhruv Sonagara	QA Trainee	Test case design, automation support, and execution

9. Schedule / Milestones

Testing follows the hackathon delivery timeline and is executed in the following sequence rather than fixed calendar dates:

Phase	Activities
Test planning & framework setup	Define scope, set up Playwright + TypeScript + Zod framework
Test case design	Author manual and automated test cases per module
Test execution — UI & Functional	Execute Hero, Login, Brief, Competitor modules
Test execution — Discovery & Run, Dashboard	Execute node-boundary, contract, and dashboard tests
API contract validation	Validate endpoints against OpenAPI spec, log gaps

Phase	Activities
Regression & sign-off	Re-run full suite, finalise reports, and obtain sign-off

10. Risks and Mitigations

Risk	Impact	Mitigation
Undocumented API behaviour (e.g. unexpected 409 conflicts)	Medium	Log as contract gap issue immediately; communicate to backend owners via Teams/GitHub
Tight hackathon timeline limiting regression coverage	Medium	Prioritise Critical/High test cases first; automate repeatable checks
Async job lifecycle (Discovery & Run) causing flaky tests	Medium	Use polling helpers with defined timeouts instead of fixed waits
Environment instability (Azure staging)	Low	Coordinate with DevOps before test execution windows
Incomplete competitor/test data for edge cases	Low	Use synthetic data to simulate limited or missing competitor sets

11. Defect Management

Defects and contract gaps discovered during testing are logged as GitHub Issues with clear reproduction steps, expected vs. actual behaviour, and severity. Each issue is triaged with the relevant contract owner (backend/DevOps) and tracked to resolution or explicit acceptance.

11.1 Severity Definitions

- Critical — blocks a core user journey (e.g. cannot sign in, cannot start analysis)
- High — a major feature misbehaves but a workaround exists
- Medium — a visible but non-blocking defect (e.g. minor layout issue)
- Low — cosmetic or edge-case issue with negligible user impact

12. Test Case Summary Reference

The table below summarises the current manual test case inventory by module. Full test case detail, steps, and pass/fail status are maintained in the companion Rivalyze Test Case Documentation.

Module / Page	Test Case Count
Hero Page	25
Hero Functional	9
Login Page	12
Login Functional	11

Module / Page	Test Case Count
Brief Page	20
Competitor Page	4
Discovery & Run Page	30
Dashboard Page	3
Total	114

13. Approval

This test plan is considered approved once signed off by the following stakeholders:

Name	Role	Signature	Date
Rushabh Sorathiya	QA Engineer		
Hely Kikani	QA Trainee		
Dhruv Sonagara	QA Trainee		